

Module 5

1 Accuracy

How often is the model correct overall?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ✓ Good when classes are **balanced** (equal positives and negatives).
- ✗ Misleading when data is **imbalanced** (e.g., 95% negative, 5% positive).

Example:

If 90 out of 100 predictions are correct → accuracy = 90%.

2 Precision

Of all the examples predicted as positive, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- ✓ High precision = very few false alarms (FP).

Used when **false positives are costly**, e.g.:

- Spam detection (you don't want to mark good emails as spam)
 - Medical tests (you don't want to tell healthy people they're sick)
-

3 Recall (Sensitivity / True Positive Rate)

Of all actual positive examples, how many did we correctly identify?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- ✓ High recall = we missed very few actual positives.

Used when **false negatives are costly**, e.g.:

- Cancer detection (missing a patient is dangerous)
 - Fraud detection
-

4 F-Measure (F1-Score)

Balances **Precision** and **Recall** into one number.

$$F1 = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

- ✓ Useful when you want a single score that balances both.
- ✓ Works well when classes are imbalanced.

⚙️ 5 Receiver Operating Characteristic (ROC) Curve

A curve that shows how the classifier's performance changes as the **decision threshold** changes.

It plots:

- **True Positive Rate (TPR)** = Recall = $\frac{TP}{TP+FN}$
- **False Positive Rate (FPR)** = $\frac{FP}{FP+TN}$

As you vary the threshold (e.g., $0.5 \rightarrow 0.6 \rightarrow 0.7$), you get different TPR/FPR pairs.

The **ROC curve** plots TPR (y-axis) vs FPR (x-axis).

- Perfect classifier $\rightarrow$ curve passes through top-left corner (TPR=1, FPR=0)
- Random classifier $\rightarrow$ diagonal line

⚙️ 6 Area Under Curve (AUC)

A single number that represents how good the ROC curve is.

$$AUC = \text{Area under the ROC curve}$$

- AUC = 1 $\rightarrow$ Perfect model
 - AUC = 0.5 $\rightarrow$ No better than random guessing
 - AUC < 0.5 $\rightarrow$ Worse than random
- ✓ Great for comparing classifiers, especially on **imbalanced data**.

Significance of ROC Curve

1 Helps visualize trade-offs

ROC shows how **recall (TPR)** increases as you tolerate more **false positives (FPR)**. This helps you choose a **threshold** depending on your problem:

- If you want **high recall** $\rightarrow$ move toward the top-right (accept more FPs)

- If you want **high precision** → stay toward the bottom-left (fewer FPs)
-

2 Compares different classifiers

By plotting multiple models on the same ROC graph, you can **compare their performance** visually.

The one **closer to the top-left corner** (higher TPR, lower FPR) is **better**.

3 Independent of threshold

Accuracy depends on a chosen cutoff (like 0.5), but ROC considers **all possible thresholds**. That makes it a **more complete evaluation** of the model.

4 Summarized by AUC (Area Under Curve)

The **AUC value** gives a single number:

- $AUC = 1$ → perfect model
 - $AUC = 0.5$ → random guessing
- So, higher AUC → better ability to distinguish between classes.
-

5 Useful for imbalanced datasets

When one class heavily outnumbers another, accuracy becomes misleading.

ROC (and AUC) focus on **true and false rates**, making them more reliable in such cases.

Bootstrapping

What it is (short):

Bootstrapping is a statistical resampling technique: repeatedly sample (with replacement) from your dataset to create many “bootstrap” datasets, then compute the statistic of interest on each sample to estimate its distribution (variance, bias, confidence intervals).

Why use it:

When you don't know the theoretical distribution of an estimator (mean, model accuracy, etc.), bootstrapping gives an empirical estimate of its sampling distribution. It's especially useful for small/complex datasets or when analytic formulas are hard.

How it works (step-by-step):

1. Given dataset D of size n .

2. Repeat B times (e.g., B = 1000):
 - Create bootstrap sample D_b by sampling n examples from D **with replacement**.
 - Compute statistic s_b (e.g., model accuracy, mean, regression coefficient) on D_b .
3. Now $\{s_1, s_2, \dots, s_B\}$ approximates the sampling distribution of the statistic.
4. Use percentiles of that distribution to form confidence intervals (e.g., 2.5th and 97.5th percentiles $\rightarrow$ 95% CI).

Key facts:

- Each bootstrap sample contains roughly 63.2% of the unique original samples on average (because probability a particular original sample is *not* drawn in n draws is $(1 - 1/n)^n \approx e^{-1} \approx 0.368$).
- **Bootstrapped SE (standard error):** $\widehat{SE} = \sqrt{\frac{1}{B-1} \sum_b (s_b - \bar{s})^2}$
- **Percentile CI:** use empirical quantiles of $\{s_b\}$
- There are improved bootstrap CI methods (e.g., BCa), but percentile is simple and common.

When to avoid / limitations:

- If data are heavily dependent (time series) you need block bootstrap variants.
- With extremely small datasets bootstrap estimates can be unstable.
- Bootstrapping assumes the sample is representative of the population.

Practical use-cases:

- Estimating confidence intervals on model accuracy, AUC, coefficients.
- Bagging (bootstrap aggregating) — see ensembles below.

Cross-Validation (CV)

What it is (short):

Cross-validation is a resampling method for **evaluating** and tuning models by splitting data into train/validation sets many times to estimate how well a model generalizes.

Why use it:

To get a more reliable estimate of test performance than a single train/test split, and to select hyperparameters without leaking test data.

Common CV types:

1. **Hold-out / Train-Test split**
 - Split once (e.g., 70/30). Simple but high variance.

2. **k-fold Cross-Validation**

- Partition data into k roughly equal folds.
- For each fold i : train on other $k-1$ folds, evaluate on fold i .
- Average the k performance scores $\rightarrow$ CV estimate.
- Typical k : 5 or 10.

3. **Stratified k-fold**

- Maintain class proportions in each fold (important for classification, especially with imbalance).

4. **Leave-One-Out CV (LOOCV)**

- $k = n$ (each fold is one sample). Low bias, high variance and computationally expensive.

5. **Repeated k-fold**

- Repeat k -fold several times with different splits and average results for more stability.

6. **Time Series CV (rolling / forward chaining)**

- Respect temporal order: train on earlier data, validate on later. Don't shuffle time series.

How to use for model selection:

- For each candidate hyperparameter setting:
 - Perform CV and get mean CV score.
- Pick hyperparameters with best mean CV score.
- Optionally retrain on full training set with chosen hyperparameters and evaluate on a held-out test set.

Bias–variance of CV estimators:

- LOOCV: low bias (uses almost all data for training) but high variance of estimate.
- k -fold ($k=5$ or 10): good bias-variance tradeoff; widely used.

Ensemble Methods

What they are (short):

Ensembles combine multiple models (learners) to produce a stronger combined predictor. The basic goal: reduce error by aggregating models that make different mistakes.

Why ensembles help:

If individual models are somewhat accurate and **make uncorrelated errors**, combining them reduces variance and/or bias — often yielding better performance than any single model.

Core concepts:

- **Diversity:** base learners should be diverse (different errors) for the ensemble to gain.
 - **Aggregation method:** majority vote (classification), average probabilities, weighted average, meta-learner (stacking).
-

Major ensemble families

1) Bagging (Bootstrap Aggregating)

- **Idea:** Train many models on bootstrap samples (sampling with replacement) and average/vote.
- **Goal:** Reduce variance (helpful for unstable learners like decision trees).
- **Algorithm (Random Forest is bagging + feature randomness):**
 - For $b = 1..B$:
 - Draw bootstrap sample D_b .
 - Train base learner (e.g., decision tree) on D_b .
 - Aggregate predictions by majority vote (classification) or average (regression).
- **Random Forest specifics:**
 - At each split in tree, choose best split among a random subset of features (mtry). This increases diversity.
 - Built-in OOB (out-of-bag) error estimate: for each sample, average predictions only from trees where the sample was *not* in that tree's bootstrap sample.
- **When to use:** when base models have high variance (deep trees).

2) Boosting

- **Idea:** Train models sequentially; each new model focuses on examples the previous ones handled poorly. Combine models as weighted sum.
- **Goal:** Reduce bias and can also reduce variance; produce a strong learner from weak learners.
- **Famous algorithms:**
 - **AdaBoost:** reweights training examples; misclassified examples get higher weight.
 - **Gradient Boosting Machines (GBM):** view boosting as gradient descent in function space. Each new learner fits the residuals (negative gradients) of the loss.
 - **XGBoost / LightGBM / CatBoost:** efficient, regularized gradient boosting implementations with many optimizations.

3) Stacking (Stacked Generalization)

- **Idea:** Train several base learners (level-0), then train a meta-learner (level-1) on their predictions to learn how to combine them.
- **How to do it safely:**
 - Use cross-validated predictions of base learners on training set to create meta-features (to avoid leakage).
 - Then fit meta-learner on those meta-features.
- **When to use:** when you want to combine heterogeneous models (SVM, tree, neural net) and let a model learn optimal blending.

4) Voting (simple ensemble)

- **Hard voting:** majority vote of class labels.
 - **Soft voting:** average predicted class probabilities and pick class with highest average probability.
 - **When to use:** simple, quick ensemble for diverse classifiers.
-

Why ensembles work (bias–variance view)

- **Bagging** reduces **variance** (averaging many unstable models). Bias $\approx$ unchanged.
 - **Boosting** primarily reduces **bias** (by sequentially correcting errors), but can also overfit if too many rounds; regularization is important.
 - **Stacking** aims to reduce both by letting meta-learner correct systematic mistakes.
-

Practical considerations & tips

- **Overfitting:** ensembles can overfit less than single complex models, but boosting can overfit if not regularized. Use early stopping and regularization.
- **Interpretability:** ensembles (especially boosting, forests) are less interpretable than a single tree. Use SHAP/LIME for explanations.
- **Computational cost:** ensembles require more compute/memory; consider model size and latency needs.
- **Hyperparameters to tune:**
 - Random Forest: number of trees, max depth, mtry (#features per split)
 - XGBoost/GBM: learning rate (eta), max_depth, subsample, colsample_bytree, number of estimators
- **When ensembles shine:** structured/tabular data, competitions, when predictive accuracy is the priority.

- **When to prefer simpler models:** when you need interpretability, low latency, or small dataset size.